

HPC & Parallel Programming

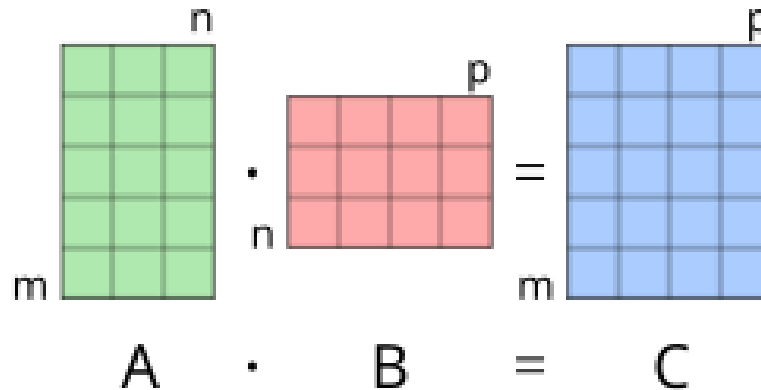
OpenMP Project Lab

Kun Suo

Computer Science, Kennesaw State University

<https://kevinsuo.github.io/>

Background - Matrix Multiplication



- Matrix multiplication is fundamentally simple but reveals remarkable depth and complexity when optimized for high-performance computing (HPC)
- Accelerating matrix multiplication is a central focus in modern computer design, especially for AI and scientific applications
- Recent hardware advancements, such as NVIDIA's H100 and B200 chips, feature Tensor Cores and Tensor Memory Accelerators engineered specifically to maximize matrix multiplication throughput



The Baseline - Sequential Matrix Multiplication

```
for(int row = 0 ; row < N ; row++){  
    for(int col = 0; col < N ; col++){  
        double resultValue = 0;  
        for(int transNumber = 0 ; transNumber < N ; transNumber++) {  
            resultValue += firstMatrix [row] [transNumber] *  
secondMatrix [transNumber] [col] ;  
        }  
        matrixMultiResult [row] [col] = resultValue;  
    }  
}
```

- The project provides a sequential base code to multiply two matrices of order $N=2048$
- Function `matrixInit()` initializes elements using a randomized double value
- Function `matrixMulti()` performs standard $O(N^3)$ multiplication using 3 nested loops (row, col, transNumber)
- Currently, the results are computed entirely sequentially on a single thread



Measuring Execution Time

- Accurate timing is critical for this lab. You have two options to measure time:
 - 1. `clock()`: Records CPU ticks.
 - ▶ Warning: When running parallel processes, CPU ticks increase multiplicatively. You must divide the `clock()` output by the number of threads to get wall time
 - 2. `omp_get_wtime()`: Returns a raw wall-clock timestamp, which safely measures absolute elapsed time regardless of thread count



```
#include <stdio.h>
#include <unistd.h>

int main ()
{
    printf("Running 'net join' with the following parameters: \n");
    char *domain="mydomain";
    char *user="domainjoinuser";
    char *pass="mypassword";
    char *vastool="/opt/quest/bin/vastool";
    char *ou="OU=Test,DC=ntdomain,DC=local";
    char unjoin[64];

    sprintf(unjoin,"%s %s -u '%s' -w '%s' unjoin -f",user,pass);

    printf("Domain: %s\n",domain);
    printf("User: %s\n",user);
    printf("-----\n");

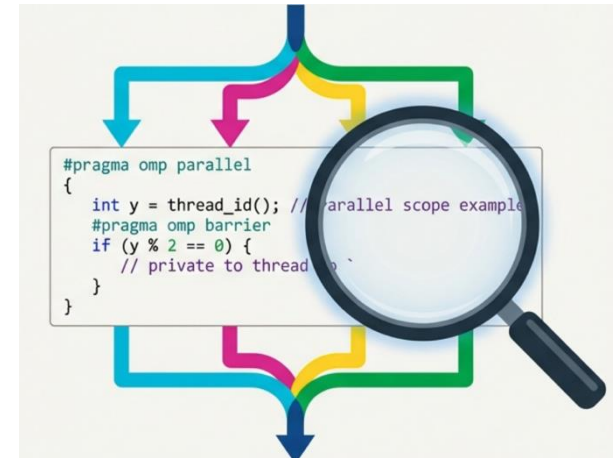
    printf("\nUnjoin ..... \n");
    system(unjoin);

    printf("\nJoin ..... \n");
    execl("/opt/quest/bin/vastool", "vastool", "-u", user, "-w", pass, "join", "-c", ou,
          "-f", domain, (char*)0);
}
```



Task 1 - OpenMP Parallelization

- Goal: Write a parallel program using OpenMP based on the sequential baseline to accelerate execution
- You must utilize OpenMP directives (such as `#pragma omp parallel for`) to distribute loop iterations across multiple CPU cores
- To compile your OpenMP-enabled C code, use the following command: `$ gcc program.c -o program.o -fopenmp`



Task 1 - Evaluation & Metrics

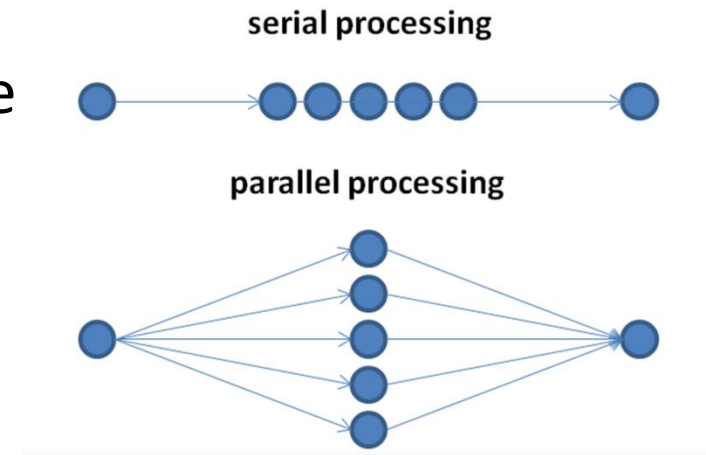
- Run your newly parallelized solution and compare it to the sequential solution
- You must test the execution time under different matrix orders: $N = 1024, 2048$, and 4096
- For accurate results, run your program multiple times and compute the average time for each execution
- Record the "Sequential Time" and "Parallel Time" in a table

Order of Matrix	1024	2048	4096
Sequential Time			
Parallel Time			
Speedup			



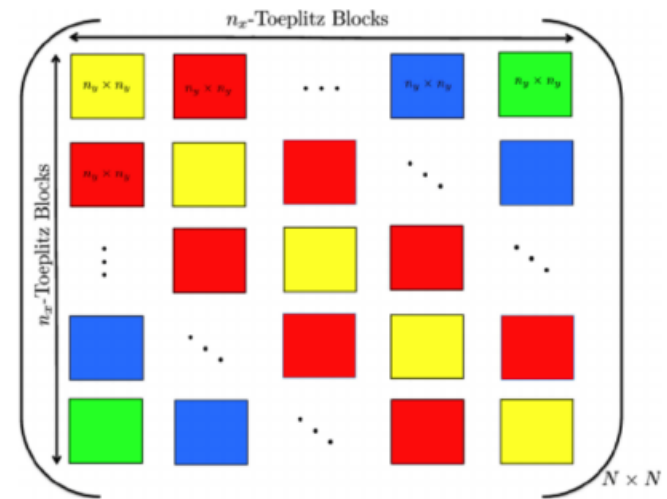
Speedup Calculation

- To measure the success of your parallelization, you must calculate the Speedup for each matrix size
- The formula for Speedup is:
$$S(n,p) = T_{\text{serial}}(n) / T_{\text{parallel}}(n,p)$$
- This ratio demonstrates how much faster the parallel OpenMP code executes compared to the original single-threaded program.



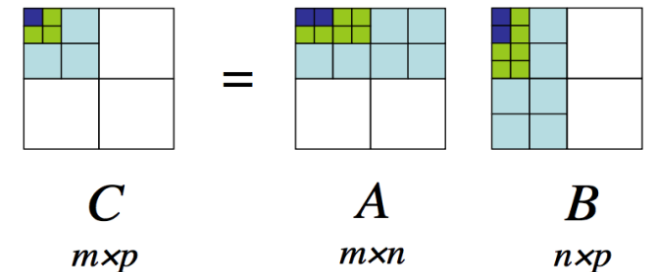
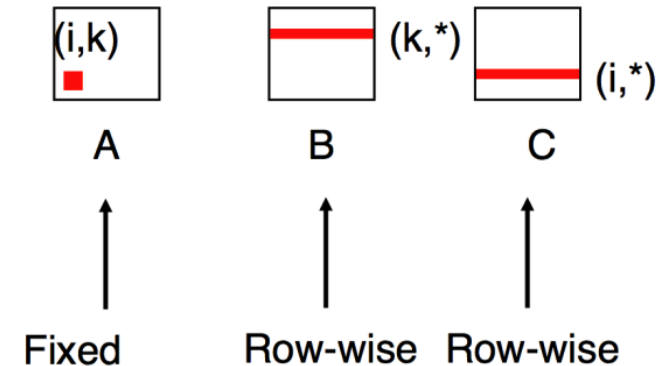
Optimizing Matrix Multiplication - Why Block?

- How can we make the parallel program run even faster? By accelerating the serial efficiency first
- Standard matrix multiplication suffers from poor cache utilization because it continuously fetches new rows and columns from main memory
- When a matrix is too large to fit in the CPU cache, the overhead of moving data drastically slows down computation



Data Locality in Matrix Multiplication

- Temporal Locality: Every inner loop calculation reuses the value of $A[i][k]$
- Spatial Locality: We can partition the massive matrix into smaller sub-matrices (blocks)
- By processing the matrix one block at a time, the CPU can keep these smaller chunks entirely inside the cache, significantly improving the cache hit rate



Task 2 - Block Optimization

- Goal: Implement a block-optimized matrix multiplication program, and then use OpenMP to parallelize it
- Instead of the standard 3 nested loops, block optimization typically uses 6 nested loops
- Outer loops stride across the matrices by BLOCK_SIZE (k2, i2, j2), while inner loops perform the standard multiplication inside each block (k, i, j)

```
for (k2 = 0; k2 < N; k2+=BLOCK_SIZE) {  
    for (i2 = 0; i2 < N; i2+=BLOCK_SIZE) {  
        for (j2 = 0; j2 < N; j2+=BLOCK_SIZE) {  
            //inside each block  
            for (k = k2; k < k2+BLOCK_SIZE; k++) {  
                for (i = i2; i < i2+BLOCK_SIZE; i++) {  
                    for (j = j2; j < j2+BLOCK_SIZE; j++) {  
                        C[i][j] += A[i][k] * B[k][j];  
                    }  
                }  
            }  
        }  
    }  
}
```



Task 2 - Evaluation & Metrics

- Apply OpenMP pragmas to your block-optimized loops
- Once completed, evaluate this implementation against the standard parallel implementation
- Test the execution times across matrix orders $N = 1024$, 2048, and 4096
- Record the "Block-optimized Sequential Time", "Block-optimized Parallel Time", and calculate the resulting Speedup



Expected Performance Hierarchy

- By the end of Task 2, you will compare three distinct configurations:
 - Single Thread (ST): Baseline sequential code
 - Using OpenMP (OMP): Standard parallelized code
 - Using OpenMP + Block (OMP-b): Parallelized and cache-optimized code
- Expected Output: ST Time > OMP Time > OMP-b Time. The blocked parallel code should execute the fastest.



Performance Analysis

- When writing your report, carefully compare the results from Task 1 and Task 2
- Address the following guiding questions in your analysis:
 - How much does OpenMP improve upon the sequential baseline?
 - How much does block-optimization improve sequential code?
 - How much does block-optimization improve on top of OpenMP?
 - What is the impact of changing the matrix size (1024 vs 2048 vs 4096)?

